

Bridge GeCode - Validador y Evaluador de Expresiones

Pipeline de Verificación para Constraint Programming

Proyecto GNUBison

2026

Contents

Introducción	2
¿Qué es Bridge GeCode?	2
Características Principales	2
Tipos de Datos Soportados	2
Operadores	2
Arquitectura del Sistema	4
Componentes Principales	4
Archivos del Proyecto	4
Compilación e Instalación	5
Requisitos	5
Instalación de Dependencias	5
Compilación	5
Verificación	5
Estructura de Directorios	5

Introducción

¿Qué es Bridge GeCode?

Bridge GeCode es un compilador y evaluador de expresiones diseñado para problemas de **Constraint Programming (CSP)**. Funciona como un pipeline que:

1. **Recibe** especificaciones en formato JSON o Pascal-like
2. **Valida** la sintaxis y semántica
3. **Evalúa** expresiones con variables
4. **Retorna** resultados en formato JSON

Características Principales

- **Validación de Sintaxis:** Parser robusto con Flex/Bison
- **Evaluación de Expresiones:** Calcula resultados automáticamente
- **Soporte Dual:** Acepta formato JSON y Pascal-like
- **Manejo de Incertidumbre:** Variables con múltiples valores posibles
- **Operaciones de Conjuntos:** UNION, INTERSECT, DIFFERENCE, SUBSET, CARDINALITY, IN
- **Sintaxis Moderna de Sets:** {a,b,c} sin comillas en expresiones
- **Salida Estructurada:** JSON con resultados detallados
- **Funciones Matemáticas:** abs, sqrt, sin, cos, ln, exp, etc.

Tipos de Datos Soportados

Tipo	Descripción	Ejemplo
integer	Números enteros	[1, 100]
float	Números decimales	[0.0, 50.0]
logic	Booleanos	[true, false]
set	Conjuntos de strings	["A", "B", "C"]

Operadores

Aritméticos

+, -, *, /

Lógicos

AND, OR, NOT, IMPLICA

Relacionales

=, <>, <, >, <=, >=

Operadores de Conjuntos

- **IN:** Verificar pertenencia $\rightarrow x \text{ IN } \{a,b,c\}$
- **UNION:** Unión de conjuntos $\rightarrow A \text{ UNION } B$

- **INTERSECT**: Intersección $\rightarrow A \text{ INTERSECT } B$
- **DIFFERENCE**: Diferencia $\rightarrow A \text{ DIFFERENCE } B$
- **SUBSET**: Verificar subconjunto $\rightarrow A \text{ SUBSET } B$
- **CARDINALITY**: Contar elementos $\rightarrow \text{CARDINALITY}(A)$

Funciones Estándar

`abs(x)`, `sqrt(x)`, `sqr(x)`, `sin(x)`, `cos(x)`, `ln(x)`, `exp(x)`

Arquitectura del Sistema

Componentes Principales



Archivos del Proyecto

- **cJSON.c**: Biblioteca JSON (MIT License)

Compilación e Instalación

Requisitos

- **gcc** (compilador C)
- **bison** ≥ 3.0
- **flex** ≥ 2.6
- **make**

Instalación de Dependencias

Ubuntu/Debian

```
sudo apt install gcc bison flex make
```

Fedora/RHEL

```
sudo dnf install gcc bison flex make
```

Compilación

Clonar o descargar el proyecto

```
cd GNUBison
```

Compilar

```
make
```

Limpiar archivos generados

```
make clean
```

Verificación

Ejecutar ejemplo simple

Debe mostrar JSON con resultados

Estructura de Directorios

GNUBison/